

Database Implementation and Indexing

Below are screenshots which prove that we implemented the database locally. Since taking the screenshots we have also implemented our database on Google Cloud Platform.

```
Last login: Thu Jul 24 08:41:02 on ttys003
[lucaseby@Mac-14 ~ % mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 47
Server version: 9.3.0 Homebrew

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

[mysql> USE srs1-fit;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
[mysql> SHOW TABLES;
+-----+
| Tables_in_srs1-fit |
+-----+
| ExerciseLog         |
| Exercises           |
| ExercisesMuscles    |
| Muscles             |
| Sets               |
| Users              |
| WorkoutContents     |
| WorkoutTemplates    |
+-----+
8 rows in set (0.003 sec)
```

Figure 1: Local Database Connection

```
mysql> DESCRIBE Exercises;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| exerciseId | int       | NO   | PRI | NULL    | auto_increment |
| ownerId    | int       | YES  | MUL | NULL    |              |
| name       | varchar(255) | NO   | MUL | NULL    |              |
| description | varchar(500) | YES  |     | NULL    |              |
| video      | varchar(255) | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.007 sec)

mysql> DESCRIBE ExerciseLog;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| userId     | int       | NO   | PRI | NULL    |              |
| exerciseId | int       | NO   | PRI | NULL    |              |
| like       | tinyint(1) | YES  | MUL | NULL    |              |
| timesCompleted | int       | YES  |     | 0       |              |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

mysql> DESCRIBE ExercisesMuscles;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| exerciseId | int       | NO   | PRI | NULL    |              |
| muscleId    | int       | NO   | PRI | NULL    |              |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)

mysql> DESCRIBE Sets;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| setId      | int       | NO   | PRI | NULL    | auto_increment |
| userId     | int       | NO   | PRI | NULL    |              |
| exerciseId | int       | NO   | PRI | NULL    |              |
| order      | int       | NO   |     | NULL    |              |
| lbs        | double    | NO   |     | NULL    |              |
| reps       | int       | NO   |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.001 sec)
```

Figure 2: Description of Exercises, ExerciseLog, ExercisesMuscles, and Sets Tables

```
mysql> DESCRIBE Muscles;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| muscleId   | int       | NO   | PRI | NULL    | auto_increment |
| name       | varchar(255) | NO   | UNI | NULL    |              |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.001 sec)

mysql> DESCRIBE Users;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| userId     | int       | NO   | PRI | NULL    | auto_increment |
| username    | varchar(30) | NO   | UNI | NULL    |              |
| password    | varchar(255) | NO   |     | NULL    |              |
| firstName   | varchar(30) | NO   |     | NULL    |              |
| lastName    | varchar(30) | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

mysql> DESCRIBE WorkoutTemplates;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| workoutId  | int       | NO   | PRI | NULL    | auto_increment |
| userId     | int       | NO   | PRI | NULL    |              |
| lastDate   | datetime  | YES  |     | NULL    |              |
| name       | varchar(255) | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.001 sec)

mysql> DESCRIBE WorkoutContents;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| workoutId  | int       | NO   | PRI | NULL    |              |
| userId     | int       | NO   | PRI | NULL    |              |
| exerciseId | int       | NO   | PRI | NULL    |              |
| order      | int       | NO   | MUL | NULL    |              |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.002 sec)
```

Figure 3: Description of Muscles, Users, WorkoutTemplates, and WorkoutContents Tables

Schema

As per the instructions, we implemented all 8 of the tables we designed in Data Definition Language. The full schema is below.

```
-- Create database to be used, Drop if already created
DROP DATABASE IF EXISTS `srsly_fit`;
CREATE DATABASE `srsly_fit`;

-- Specify which database to use
USE `srsly_fit`;

-- DROP TABLE statements are now in reverse order of creation to avoid foreign key constraint
errors
DROP TABLE IF EXISTS `Sets`;
DROP TABLE IF EXISTS `ExerciseLog`;
DROP TABLE IF EXISTS `WorkoutContents`;
DROP TABLE IF EXISTS `ExercisesMuscles`;
DROP TABLE IF EXISTS `Exercises`;
DROP TABLE IF EXISTS `WorkoutTemplates`;
DROP TABLE IF EXISTS `Muscles`;
DROP TABLE IF EXISTS `Users`;

-- Users table created first as many other tables depend on it.
CREATE TABLE `Users` (
  -- Added AUTO_INCREMENT to automatically generate unique user IDs.
  `userId` INT PRIMARY KEY AUTO_INCREMENT,
  `username` VARCHAR(30) UNIQUE NOT NULL,
  -- Increased password length to 255 to accommodate hashed passwords, which is a security
  best practice.
  `password` VARCHAR(255) NOT NULL,
  `firstName` VARCHAR(30) NOT NULL,
  `lastName` VARCHAR(30) NULL
);

-- Muscles table has no dependencies.
CREATE TABLE `Muscles` (
  -- Added AUTO_INCREMENT.
  `muscleId` INT PRIMARY KEY AUTO_INCREMENT,
  `name` VARCHAR(255) UNIQUE NOT NULL
);

-- WorkoutTemplates depends on Users.
CREATE TABLE `WorkoutTemplates` (
  -- Added AUTO_INCREMENT
```

```

`workoutId` INT AUTO_INCREMENT,
`userId` INT,
`lastDate` DATETIME NULL,
`name` VARCHAR(255),
-- Added both id's as primary key because it's a weak entity that's tied to the user
PRIMARY KEY(workoutId, userId),
FOREIGN KEY (`userId`) REFERENCES `Users`(`userId`) ON DELETE CASCADE
);

-- Exercises depends on Users.
CREATE TABLE `Exercises` (
-- Added AUTO_INCREMENT.
`exercisId` INT PRIMARY KEY AUTO_INCREMENT,
`ownerId` INT DEFAULT NULL,
`name` VARCHAR(255) NOT NULL,
-- Renamed `desc` to `description` because DESC is a reserved SQL keyword.
`description` TEXT NULL,
`video` VARCHAR(255) NULL,
-- Switched ON DELETE SET NULL with ON DELETE CASCADE:
-- If a user is deleted, their custom exercises should also be deleted.
-- We can't set the exercises to NULL because we use that to specify whether an exercise is
default.
FOREIGN KEY (`ownerId`) REFERENCES `Users`(`userId`) ON DELETE CASCADE
);
-- This is a "join table" or "linking table". It depends on Exercises and Muscles.
CREATE TABLE `ExercisesMuscles` (
`exercisId` INT,
`muscleId` INT,
PRIMARY KEY (`exercisId`, `muscleId`),
-- Added ON DELETE CASCADE: If an exercise or muscle is deleted, the corresponding link
is also deleted.
FOREIGN KEY (`exercisId`) REFERENCES `Exercises`(`exercisId`) ON DELETE
CASCADE,
FOREIGN KEY (`muscleId`) REFERENCES `Muscles`(`muscleId`) ON DELETE CASCADE
);
-- Depends on WorkoutTemplates and Exercises.
CREATE TABLE `WorkoutContents` (
`workoutId` INT,
`userId` INT,
`exercisId` INT,
-- `order` is a reserved keyword, so it's good practice to wrap it in backticks.
`order` INT NOT NULL,
PRIMARY KEY (`workoutId`, `userId`, `exercisId`),
FOREIGN KEY (`workoutId`, `userId`) REFERENCES `WorkoutTemplates`(`workoutId`,

```

```
`userId`) ON DELETE CASCADE,
  FOREIGN KEY (`exerciseld`) REFERENCES `Exercises`(`exerciseld`) ON DELETE
  CASCADE
);
```

-- Depends on Users and Exercises.

```
CREATE TABLE `ExerciseLog` (
  `userId` INT,
  `exerciseld` INT,
  -- `like` is a reserved keyword, so it's wrapped in backticks.
  `like` BOOL NULL,
  `timesCompleted` INT DEFAULT 0,
  PRIMARY KEY (`userId`, `exerciseld`),
  FOREIGN KEY (`userId`) REFERENCES `Users`(`userId`) ON DELETE CASCADE,
  FOREIGN KEY (`exerciseld`) REFERENCES `Exercises`(`exerciseld`) ON DELETE
  CASCADE
);
```

-- Depends on ExerciseLog.

```
CREATE TABLE `Sets` (
  -- Added AUTO_INCREMENT
  `setId` INT AUTO_INCREMENT,
  `userId` INT,
  `exerciseld` INT,
  `order` INT NOT NULL,
  `lbs` REAL NOT NULL,
  `reps` INT NOT NULL,
  -- The primary key is made up of the combination of all 3 because
  -- this is a weak entity relationship with the ExerciseLog record
  PRIMARY KEY(setId, userId, exerciseld),
  FOREIGN KEY (`userId`, `exerciseld`) REFERENCES `ExerciseLog`(`userId`, `exerciseld`)
  ON DELETE CASCADE
);
```

Inserting Data

We created csv files for each table and imported them locally and later on Google Cloud Platform. We were able to pull most of our data from the Gym Exercise Dataset on Kaggle. For the tables where we did not have data, we used Excel commands to generate the rest of the data when possible. As a last resort, we used auto-generated data to fill in some of the gaps in the gaps in the missing data to adequately fill all of the tables. We implemented all 8 of our tables and imported over 1000 rows of data into the Exercises, ExercisesMuscles, ExerciseLog, and Sets tables. The size of each table is below.

```

mysql> SELECT COUNT(*) FROM Exercises;
+-----+
| COUNT(*) |
+-----+
|      2920 |
+-----+
1 row in set (0.001 sec)

mysql> SELECT COUNT(*) FROM ExerciseLog;
+-----+
| COUNT(*) |
+-----+
|      5838 |
+-----+
1 row in set (0.002 sec)

mysql> SELECT COUNT(*) FROM ExercisesMuscles;
+-----+
| COUNT(*) |
+-----+
|      2918 |
+-----+
1 row in set (0.001 sec)

mysql> SELECT COUNT(*) FROM Sets;
+-----+
| COUNT(*) |
+-----+
|      5863 |
+-----+
1 row in set (0.001 sec)

mysql> SELECT COUNT(*) FROM Muscles;
+-----+
| COUNT(*) |
+-----+
|        17 |
+-----+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) FROM Users;
+-----+
| COUNT(*) |
+-----+
|         2 |
+-----+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) FROM WorkoutTemplates;
+-----+
| COUNT(*) |
+-----+
|        16 |
+-----+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) FROM WorkoutContents;
+-----+
| COUNT(*) |
+-----+
|        86 |
+-----+
1 row in set (0.001 sec)

```

Figure 4: Number of Rows in Each Table

Workout Page Creation Advanced Query

SET @workoutId = 1;

SET @userId = 1;

```

SELECT exers.exerciseld, exers.name, eLog.`like`, conts.`order`,
       JSON_ARRAYAGG(
         JSON_OBJECT('setld', s.setld, 'order', s.`order`, 'lbs', s.lbs, 'reps', s.reps)
       ) AS sets
FROM WorkoutContents conts
LEFT JOIN Exercises exers ON exers.exerciseld = conts.exerciseld
LEFT JOIN ExerciseLog eLog ON eLog.userId = conts.userId AND eLog.exerciseld =
exers.exerciseld
LEFT JOIN (SELECT setld, userId, exerciseld, `order`, lbs, reps
           FROM Sets
           ORDER BY `order`) s
       ON s.userId = eLog.userId AND s.exerciseld = eLog.exerciseld
WHERE conts.userId = @userId AND conts.workoutId = @workoutId
GROUP BY exers.exerciseld, exers.name, eLog.`like`, conts.`order`
ORDER BY conts.`order`
LIMIT 15;

```

The above query will be used to build a “Workout Page” from a pre-created “Workout Template”. The Workout Page is a page that will be displayed to the user when they are working out, it will show all of the exercises they will be performing in order. Under each exercise, it will additionally display the set information (weight to be lifted in lbs and number of times or reps to perform the exercise) in order. This query meets the advanced specifications due to requiring multiple table joins and the requiring a GROUP BY clause so we can put all of the sets for each exercise into a JSON array to make the output easier for the front-end to process. Below is an

example output of the query. As you can see, less than 15 rows are showing because the workout the user has chosen to perform only contains 7 exercises (so 7 rows).

exerciseld	name	like	order	sets
1782	Single-leg depth s...	1	0	[{"lbs": 15.0, "reps": 18, "order": 0, "setid": 3563}, {"lbs": 20.0, "reps": 15, "order": 1, "setid": 3564}, {"lbs": 25.0, "reps": 13, "order": 2, "setid": 3565}]
1785	Forward Band Walk	1	1	[{"lbs": 50.0, "reps": 10, "order": 0, "setid": 3571}, {"lbs": 50.0, "reps": 17, "order": 1, "setid": 3572}, {"lbs": 50.0, "reps": 6, "order": 2, "setid": 3573}]
1784	Hip Flexion with B...	1	2	[{"lbs": 35.0, "reps": 13, "order": 0, "setid": 3569}, {"lbs": 35.0, "reps": 9, "order": 1, "setid": 3570}]
1781	Scapular Pull-Up	0	3	[{"lbs": 20.0, "reps": 7, "order": 0, "setid": 3561}, {"lbs": 25.0, "reps": 5, "order": 1, "setid": 3562}]
1788	Speed Box Squat	1	4	[{"lbs": 15.0, "reps": 19, "order": 0, "setid": 3579}, {"lbs": 20.0, "reps": 18, "order": 1, "setid": 3580}]
1783	Squats - With Bands	1	5	[{"lbs": 20.0, "reps": 11, "order": 0, "setid": 3566}, {"lbs": 25.0, "reps": 10, "order": 1, "setid": 3567}, {"lbs": 25.0, "reps": 6, "order": 2, "setid": 3568}]
1790	UP Band Abducted...	1	6	[{"lbs": 10.0, "reps": 16, "order": 0, "setid": 3583}, {"lbs": 15.0, "reps": 13, "order": 1, "setid": 3584}]

Figure 5: Workout Page Creation Query Example Output

```
mysql> EXPLAIN ANALYZE
-> SELECT exers.exerciseld, exers.name, eLog.`like`, conts.`order`, JSON_ARRAYAGG(JSON_OBJECT('setid', s.setid, 'order', s.`order`, 'lbs', s.lbs, 'reps', s.reps)) AS sets
-> FROM WorkoutContents conts
-> LEFT JOIN Exercises exers ON exers.exerciseld = conts.exerciseld
-> LEFT JOIN ExerciseLog eLog ON eLog.userid = conts.userid AND eLog.exerciseld = exers.exerciseld
-> LEFT JOIN (SELECT setid, userid, exerciseld, `order`, lbs, reps
-> FROM Sets
Display all 823 possibilities? (y or n)
-> ORDER BY `order`) s
-> ON s.userid = eLog.userid AND s.exerciseld = eLog.exerciseld
-> WHERE conts.userid = @userid AND conts.workoutid = @workoutid
-> GROUP BY exers.exerciseld, exers.name, eLog.`like`, conts.`order`
-> ORDER BY conts.`order`
-> LIMIT 15;

| EXPLAIN
+-----+
|
+-----+

| -> Limit: 15 row(s) (actual time=1.75..1.76 rows=7 loops=1)
| -> Sort: conts.`order`, limit input to 15 row(s) per chunk (actual time=1.75..1.76 rows=7 loops=1)
| -> Stream results (actual time=1.62..1.72 rows=7 loops=1)
| -> Group aggregate: json_arrayagg(tmp_field) (actual time=0.486..0.537 rows=7 loops=1)
| -> Sort: exers.exerciseld, exers.name, eLog.`like`, conts.`order` (actual time=0.476..0.48 rows=17 loops=1)
| -> Stream results (cost=15.9 rows=14.1) (actual time=0.369..0.451 rows=17 loops=1)
| -> Nested loop left join (cost=15.9 rows=14.1) (actual time=0.880..0.141 rows=17 loops=1)
| -> Nested loop left join (cost=11 rows=7) (actual time=0.8449..0.8691 rows=7 loops=1)
| -> Nested loop left join (cost=8.53 rows=7) (actual time=0.8485..0.8545 rows=7 loops=1)
| -> Index lookup on conts using PRIMARY (workoutid = (@workoutid), userid = (@userid)) (cost=0.952 rows=7) (actual time=0.8216..0.8232 rows=7 loops=1)
| -> Single-row index lookup on exers using PRIMARY (exerciseld = conts.exerciseld) (cost=0.996 rows=1) (actual time=0.80421..0.80423 rows=1 loops=7)
| -> Single-row index lookup on eLog using PRIMARY (userid = (@userid), exerciseld = exers.exerciseld) (cost=0.264 rows=1) (actual time=0.00192..0.00195 rows=1 loops=7)
| -> Index lookup on Sets using idx_sets_user_exercise_order (userid = eLog.userid, exerciseld = eLog.exerciseld) (cost=0.533 rows=2.02) (actual time=0.00891..0.0099 rows=2.43 loops=7)
|
+-----+
1 row in set (0.006 sec)
```

Figure 6: EXPLAIN ANALYZE Output of Workout Page Creation Query

Action	Cost	Rows
Stream results	15.9	14.1
Nested loop left join 1	15.9	14.1
Nested loop left join 2	11	7
Nested loop left join 3	8.53	7
Index lookup on conts using PRIMARY	0.952	7

Single-row index lookup on exers using PRIMARY	0.996	1
Single-row index lookup on eLog using PRIMARY	0.264	1
Index lookup on Sets using userId	0.533	2.02

Table 1: Initial Performance

The first left join on the Exercises table had the highest cost (15.9). Adding an index would not have improved the performance because the join was on the Exercises table's primary key. The second left join on the ExerciseLog table had the second highest cost (11). Since the join was on both of the table's primary keys, adding an index wouldn't have improved the performance. As an experiment we tried adding an index anyways and were able to confirm that the cost was unchanged. The final left join on the Sets table, with a cost of 8.53, was also a join on two of it's primary keys so adding an index would not have directly improved the left join performance. We noticed that the Sets table's rows were ordered on the order attribute prior to the data being joined to any other table, however, so we tried to add an the following index with the thought process that maybe if indexing was done with the order attribute included the ordering's cost might be decreased. So we temporarily added an index called 'idx_sets_user_exercise_order' which was on the userId, exerciseld, and order attributes of the Sets table. Upon testing our theory we found that the 'idx_sets_user_exercise_order' index did not improve performance so it was removed. In the WHERE clause, the userId and workoutId attributes of the WorkoutContents table are used to filter the results. We tried to add an index lookup which was based off of only those attributes but the cost was unchanged. We believe this is because those attributes are 2 out of 3 of the WorkoutContents table's composite primary keys. Since the results are ordered and grouped by the order attribute of the WorkoutContents table, we temporarily tried to make an index with this attribute but the cost results were unchanged. Since the query was grouped by the exerciseld and name attributes of the Exercises table, we tried to make an index made up of both attributes. Upon testing the newly created index, the costs were unchanged so we removed it. Finally, since the query was grouped by the like attribute of the ExerciseLog table, we created a temporary index with it as well but since there wasn't any change to the cost we removed it. There is a chance that this may have improved the performance of our query by a small margin if we had a larger dataset but given that the WorkoutContents table has only 83 rows we were unable to see a change in the cost. We additionally tried to change the workoutId and userId that were set before the query but were still unable to find an index that improved the performance of this query. We think these results are mainly due to two reasons: many of the things that could have been improved by adding indexes were not improved because many of the attributes in question were primary keys and the size of our dataset likely negatively affected the results. While the largest table associated with the query had over 4,000 rows (the Exercises table), adding indexes likely did not improve the performance of the query because the WorkoutContents table only had 83 rows and only at most 7 exercises were associated with the row that was chosen.

Exercise Search Bar Query

```
SET @keywords = 'push up';
```

```
SET @ownerId = 1;
```

```
SET @muscleIds = '1,3,5'; -- multiple muscle IDs separated by commas
```

```
SELECT exers.exerciseld, exers.name, JSON_ARRAYAGG(musc.name) AS muscles
```

```
FROM Exercises exers
```

```
JOIN ExercisesMuscles ems ON ems.exerciseld = exers.exerciseld
```

```
JOIN Muscles musc ON ems.muscleld = musc.muscleld
```

```
WHERE (exers.ownerId IS NULL OR exers.ownerId = @ownerId)
```

```
      AND (exers.name REGEXP REPLACE(@keywords, ' ', '|')
```

```
            OR exers.name REGEXP REPLACE(@keywords, ' ', '|'))
```

```
      AND FIND_IN_SET(musc.muscleld, @muscleIds)
```

```
GROUP BY exers.exerciseld, exers.name
```

```
LIMIT 15;
```

With the above query we implemented a keywords search that would split the keywords by the spaces between them and compare the keywords against the name and description attributes of the rows in the Exercises table to filter the returned results. We filtered the results so only exercises associated with the chosen muscleId's would be shown. Finally, we filtered the results to include only the exercises the user has access to.

exerciseld	name	muscles
6	Decline band press sit-up	["Abdominals"]
11	Decline bar press sit-up	["Abdominals"]
19	Decline plate sit-up	["Abdominals"]
32	Holman Kettlebell Up to Elbow Down	["Abdominals"]
37	FYR2 Kettlebell Get-Up Reach	["Abdominals"]
49	Dumbbell Fix Dumbbell Sit-Up	["Abdominals"]
62	Dumbbell Get-Up Sit-Up	["Abdominals"]
65	Holman Boat with Feet Push-Out and Overhe...	["Abdominals"]
77	MetaBurn Alternating Plank Row With Push-Up	["Abdominals"]
85	FYR2 Dumbbell Sit-Up Reach	["Abdominals"]
95	Otis-Up	["Abdominals"]
98	Decline bar press sit-up	["Abdominals"]
100	Weighted Sit-Ups - With Bands	["Abdominals"]
103	Barbell press sit-up	["Abdominals"]
111	Decline plate sit-up twist	["Abdominals"]

Figure 7: Initial Exercise Search Bar Query Example Output

Action	Cost	Rows
Limit: 15 row(s)	2817	15
Group aggregate: json_arrayagg(musc.`name`)	2817	2641
Nested loop inner join	2208	2641
Nested loop inner join	1284	2641
Sort: exers.exerciseld, exers.`name`	359	2641
Filter: (((exers.ownerId is null) or (exers.ownerId = <cache>((@ownerId)))) and (regexp_like(exers.`name`,<cache>(replace((@keywords)," "," ")) or regexp_like(exers.`description`, <cache>(replace((@keywords), " "," "))))))	359	2641
Table scan on exers	359	2641
Filter: (0 <> find_in_set(ems.muscleld,<cache>((@musclelds))))	0.25	1
Covering index lookup on ems using PRIMARY	0.25	1
Single-row index lookup on musc using PRIMARY	0.25	1

Table 2: Initial Performance

From analyzing the initial results, we found that there were significant costs associated with sorting and filtering based on the Exercises name and description attributes and scanning the Exercises table. Therefore, in order to optimize the query, we first added a FULLTEXT index to speed up the name attribute search and comparison in the WHERE clause. We accomplished this by adding a FULLTEXT index on the name and description attributes of the Exercises table to try to speed up the associated costs. With the addition of this index, the costs decreased drastically in most areas.

Action	Cost	Rows
Stream results	1.78	1
Nested loop inner join	1.78	1
Nested loop inner join	1.43	1
Filter: (((exers.ownerId is null) or (exers.ownerId = <cache>((@ownerId)))) and (match exers.`name`,exers.`description` against ((@keywords	1.08	1
Full-text index search on exers using ft_name_desc	1.08	1
Filter: (0 <> find_in_set(ems.muscleId,<cache>((@muscleIds))))	0.35	1
Covering index lookup on ems using PRIMARY (exercisId = exers.exercisId)	0.35	1
Single-row index lookup on musc using PRIMARY (muscleId = ems.muscleId)	0.35	1
Stream results	1.78	1

Table 3: Indexed Performance

The original query had the unfortunate problem of being unable to order the results based on the best match of what was searched. With the addition of the FULLTEXT index, we were able to modify the query to order the results based on best match with the match function to fix the issue. We additionally swapped the REGEX line with the MATCH function in the WHERE clause since MySQL interprets non-zero numbers as true with the assumption the

compiler would simplify the result. Due to these changes, we have included the modified query and FULLTEXT index below:

```
ALTER TABLE Exercises ADD FULLTEXT KEY ft_name_desc (name, description);

SET @keywords = 'push up';
SET @ownerId = 1;
SET @muscleIds = '1,3,5'; -- multiple muscle IDs separated by commas

SELECT exers.exerciseld, exers.name, JSON_ARRAYAGG(musc.name) AS muscles
FROM Exercises exers
JOIN ExercisesMuscles ems ON ems.exerciseld = exers.exerciseld
JOIN Muscles musc ON ems.muscleld = musc.muscleld
WHERE (exers.ownerId IS NULL OR exers.ownerId = @ownerId)
    AND FIND_IN_SET(musc.muscleld, @muscleIds)
    AND MATCH (exers.name, exers.description) AGAINST (@keywords)
GROUP BY exers.exerciseld, exers.name
ORDER BY MATCH (exers.name, exers.description) AGAINST (@keywords) DESC LIMIT
15;
```

With the results ordered by best match, the results that were returned changed as well so now the top 15 matches are returned. Finally, we tried to add an additional index on the Exercises table that only used the name attribute but upon analyzing the costs we found they were unchanged so we removed the index.

exerciseld	name	muscles
432	Push-up to side plank	["Abdominals"]
298	Bear crawl push-up	["Abdominals"]
323	Plank Push-Up	["Abdominals"]
282	FYR Push-Up Plank Hold	["Abdominals"]
238	Exercise ball knee roll-in	["Abdominals"]
256	Shoulder tap	["Abdominals"]
259	Sprawl	["Abdominals"]
280	FYR Sprawl	["Abdominals"]
234	Skin The Cat To Push-Up	["Abdominals"]
129	Holman Plate In-and-Out Push-Up	["Abdominals"]
302	Straight-arm plank	["Abdominals"]
303	Plank up-down	["Abdominals"]
312	Push-Up Roll Out - Gethin Variation	["Abdominals"]
77	MetaBurn Alternating Plank Row With Push-Up	["Abdominals"]
330	Prison Push-Up	["Abdominals"]

Figure 9: Exercise Search Bar Query Example Output Ordered By Best Match

```

mysql> EXPLAIN ANALYZE
-> SELECT exers.exerciseId, exers.name, JSON_ARRAYAGG(musc.name) AS muscles
-> FROM Exercises exers
-> JOIN ExercisesMuscles oms ON oms.exerciseId = exers.exerciseId
-> JOIN Muscles musc ON oms.muscleId = musc.muscleId
-> WHERE (exers.ownerId IS NULL OR exers.ownerId = @ownerId)
->    AND FIND_IN_SET(musc.muscleId, @muscleIds)
->    AND MATCH (exers.name, exers.description) AGAINST (@keywords)
-> GROUP BY exers.exerciseId, exers.name
-> ORDER BY MATCH (exers.name, exers.description) AGAINST (@keywords) DESC
-> LIMIT 15;

+-----+
| EXPLAIN |
+-----+

+-----+
| -> Limit: 15 row(s) (actual time=1.73..1.73 rows=15 loops=1)
  -> Sort: (match exers.`name`,exers.`description` against ((@keywords)))' DESC, limit input to 15 row(s) per chunk (actual time=1.72..1.73 rows=15 loops=1)
    -> Stream results (actual time=1.67..1.71 rows=33 loops=1)
      -> Group aggregate: json_arrayagg(muscles.`name`) (actual time=1.66..1.68 rows=33 loops=1)
        -> Sort: exers.exerciseId, exers.name (actual time=1.65..1.68 rows=33 loops=1)
          -> Stream results (cost=1.78 rows=1) (actual time=0.431..1.61 rows=33 loops=1)
            -> Nested loop inner join (cost=1.78 rows=1) (actual time=0.428..1.59 rows=33 loops=1)
              -> Nested loop inner join (cost=1.43 rows=1) (actual time=0.416..1.57 rows=33 loops=1)
                -> Filter: (((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) and (match exers.`name`,exers.`description` against ((@keywords)))) (cost=1.08 rows=1) (actual time=0.14..0.888 rows=283 loops=1)
                  -> Full-text index search on exers using ft_name_desc (name = (@keywords)) (cost=1.08 rows=1) (actual time=0.137..0.851 rows=284 loops=1)
                    -> Filter: (@ <> find_in_set(oms.muscleId,<cache>(@muscleIds))) (cost=0.35 rows=1) (actual time=0.00312..0.00322 rows=0.163 loops=203)
                      -> Covering index lookup on oms using PRIMARY (exerciseId = exers.exerciseId) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
                        -> Single-row index lookup on musc using PRIMARY (muscleId = oms.muscleId) (cost=0.35 rows=1) (actual time=477e-6..515e-6 rows=1 loops=33)
                -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
              -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
            -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
          -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
        -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
      -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
    -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
  -> Filter: ((exers.ownerId is null) or (exers.ownerId = <cache>(@ownerId)))) (cost=0.35 rows=1) (actual time=0.00234..0.0028 rows=1 loops=203)
+-----+
1 row in set (0.005 sec)

```

Figure 10: Final EXPLAIN ANALYZE Output of Exercise Search Bar Query

Suggest Random Exercise Advanced Query

```
-- Specify the database to use
USE `srsly_fit`;

-- Set the input parameters for the query
SET @userId = 1;
SET @workoutId = 1;

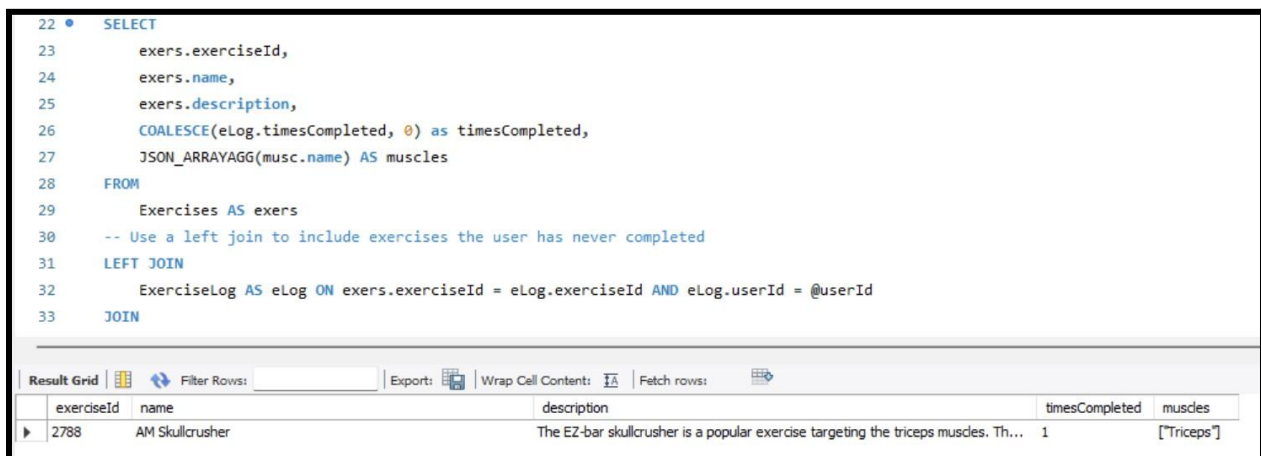
SELECT
    exers.exerciseld,
    exers.name,
    COALESCE(eLog.timesCompleted, 0) AS completed_count,
    JSON_ARRAYAGG(musc.name) AS muscles
FROM
    Exercises AS exers
LEFT JOIN
    ExerciseLog AS eLog ON exers.exerciseld = eLog.exerciseld AND eLog.userId = @userId
JOIN
    ExercisesMuscles AS ems ON exers.exerciseld = ems.exerciseld
JOIN
    Muscles AS musc ON ems.muscleld = musc.muscleld
WHERE
    (exers.ownerId IS NULL OR exers.ownerId = 1)
    AND exers.exerciseld NOT IN (
        SELECT exerciseld
        FROM WorkoutContents
        WHERE userId = @userId AND workoutId = @workoutId
    )
GROUP BY
    exers.exerciseld, exers.name, completed_count HAVING
    completed_count <= COALESCE((
        SELECT AVG(timesCompleted)
        FROM ExerciseLog
        WHERE userId = @userId
    ), 0)
ORDER BY
    RAND()
LIMIT 1;
```

With this advanced query, we can suggest a random exercise to the user based on some simple logic. Based on the past exercises a user has completed or has never attempted, we'll suggest a new random exercise to the user if the user has completed that exercise less than or equal to the average number of times they've performed all of their exercises. This way, the user will be

able to have a more rounded workout and practice exercises they might be neglecting, simply forgot about, or have never tried before.

We accomplish this by grabbing the current `userId` and `workoutId` passed to the sql script. From there, we perform a left join on the exercise log table, merging on `exerciseId` and `userId`, to get all the exercises a user has or has not completed before. Additionally, we merge on the `ExercisesMuscles` table to get the list of muscle id's associated with the exercise we'll be suggesting. Finally, we join the muscles table to get the muscle names to be aggregated into a `JSON_ARRAYAGG` object in case there are multiple muscles being activated by a single exercise.

After that, we perform some basic filtering with the `WHERE` clause to ensure we're including default exercises and the exercises that have been created by the user we're generating a random exercise for. We make sure to exclude the `exerciseId`'s that are in the current workout with the `NOT IN` keywords followed by a subquery grabbing the list of `exerciseId`'s from the current `workoutId`. Grouping by the 4 main keys from the top most select statement, we'll ensure the results are distinct by those fields. We add the `HAVING` statement with a subquery to implement the average workout time completion less than or equal to the user's average exercise times completed count. Lastly, we add the `ORDER BY` and `RAND()` keywords to ensure we're getting a new random exercise each time and `LIMIT` the results to 1 row to give the users a single randomly suggested exercise.



```
22 • SELECT
23     exers.exerciseId,
24     exers.name,
25     exers.description,
26     COALESCE(eLog.timesCompleted, 0) as timesCompleted,
27     JSON_ARRAYAGG(musc.name) AS muscles
28 FROM
29     Exercises AS exers
30 -- Use a left join to include exercises the user has never completed
31 LEFT JOIN
32     ExerciseLog AS eLog ON exers.exerciseId = eLog.exerciseId AND eLog.userId = @userId
33 JOIN
```

exerciseId	name	description	timesCompleted	muscles
2788	AM Skullcrusher	The EZ-bar skullcrusher is a popular exercise targeting the triceps muscles. Th...	1	["Triceps"]

Figure 11: Sample results from the suggest random exercise query
(Note: there is only one row because we are choosing to not overwhelm the user with choice and instead give them a single exercise to keep their workout moving)

Suggest Random Exercise Indexing Analysis

Baseline (no indexing):


```

mysql> EXPLAIN ANALYZE
--> SELECT
-->   exers.exerciseId,
-->   exers.name
--> COALESCE(eLog.timesCompleted, 0) AS completed_count,
--> JSON_ARRAYAGG(musc.name) AS muscles
--> FROM
-->   Exercises AS exers
--> LEFT JOIN
-->   ExerciseLog AS eLog ON exers.exerciseId = eLog.exerciseId AND eLog.userId = 1
--> JOIN
-->   ExercisesMuscles AS ems ON exers.exerciseId = ems.exerciseId
--> JOIN
-->   Muscles AS musc ON ems.muscleId = musc.muscleId
--> WHERE
-->   (exers.ownerId IS NULL OR exers.ownerId = 1)
--> AND exers.exerciseId NOT IN (
-->   SELECT exerciseId
-->     FROM WorkoutContents
-->    WHERE userId = 1 AND workoutId = 1
--> )
--> GROUP BY
-->   exers.exerciseId, exers.name, completed_count
--> HAVING
-->   completed_count <= COALESCE((
-->     SELECT AVG(timesCompleted)
-->     FROM ExerciseLog
-->    WHERE userId = 1
--> ), 0)
--> ORDER BY
-->   RAND()
--> LIMIT 1;

+-----+
| EXPLAIN |
+-----+

+-----+
| limit: 1 row(s) (actual time=7.41..7.41 rows=1 loops=1) |
| Sort: 'rand()' (actual time=7.41..7.41 rows=1 loops=1) |
| Filter: (completed_count <= cache(coalesce(select #3),0)) (actual time=5.88..7.29 rows=1192 loops=1) |
| Stream results (actual time=5.54..6.81 rows=2911 loops=1) |
| Group aggregate: json_arrayagg(muscles.name') (actual time=5.54..6.2 rows=2911 loops=1) |
| Sort: exers.exerciseId, exers.name, completed_count (actual time=5.53..5.65 rows=2911 loops=1) |
| Stream results (cost=6378 rows=20198) (actual time=0.8397..4.61 rows=2911 loops=1) |
| Nested loop left join (cost=6378 rows=20198) (actual time=0.8372..4.19 rows=2911 loops=1) |
| Nested loop anti-join (cost=3629 rows=20198) (actual time=0.835..2.85 rows=2911 loops=1) |
| Nested loop inner join (cost=1320 rows=2885) (actual time=0.8272..2.23 rows=2918 loops=1) |
| Nested loop inner join (cost=299 rows=2918) (actual time=0.8215..0.411 rows=2918 loops=1) |
| Covering index scan on musc using name (cost=1.95 rows=17) (actual time=0.8085..0.9131 rows=17 loops=1) |
| Covering index lookup on ems using muscleId (muscleId=musc.muscleId) (cost=1.32 rows=172) (actual time=0.80527..0.8183 rows=172 loops=17) |
| Filter: (exers.ownerId is null) or (exers.ownerId = 1) (cost=0.25 rows=0.989) (actual time=0.8076..0.549e+6 rows=1 loops=2918) |
| Single-row index lookup on exers using PRIMARY (exerciseId=ems.exerciseId) (cost=0.25 rows=1) (actual time=0.442e-6..4.456e-6 rows=1 loops=2918) |
| Single-row index lookup on <subquery2> using <auto_distinct_key> (exerciseId=ems.exerciseId) (cost=1.75..1.75 rows=1) (actual time=145e-6..145e-6 rows=0.0024 loops=2918) |
| Materialize with deduplication (cost=1.65..1.65 rows=7) (actual time=0.80671..0.80671 rows=7 loops=1) |
| Filter: (WorkoutContents.exerciseId is not null) (cost=0.954 rows=7) (actual time=0.80222..0.80428 rows=7 loops=1) |
| Covering index lookup on WorkoutContents using PRIMARY (workoutId=1, userId=1) (cost=0.954 rows=7) (actual time=0.80195..0.80374 rows=7 loops=1) |
| Index lookup on ExerciseLog using PRIMARY (userId=1, exerciseId=ems.exerciseId) (cost=0.25 rows=1) (actual time=376e-6..388e-6 rows=1 loops=2911) |
| Select #3 (subquery in condition, run only once) (cost=585 rows=1) (actual time=0.326..0.326 rows=1 loops=1) |
| Aggregate: avg(ExerciseLog.timesCompleted) (cost=585 rows=1) (actual time=0.326 rows=1 loops=1) |
| Index lookup on ExerciseLog using PRIMARY (userId=1) (cost=293 rows=2918) (actual time=0.8215..0.235 rows=2918 loops=1) |

```

Overall cost = 6370

Scenario 1: CREATE INDEX idx_exercises_owner ON Exercises(ownerId);

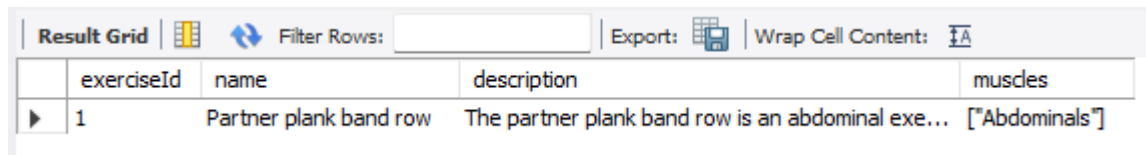
[illegible]

Cost = 6370 (no change)

Exercise Page Query

```
SELECT exers.exerciseld, exers.name, exers.description, JSON_ARRAYAGG(musc.name) AS
muscles
FROM Exercises exers
JOIN ExercisesMuscles ems ON ems.exerciseld = exers.exerciseld
JOIN Muscles musc ON ems.muscleld = musc.muscleld
WHERE (exers.ownerId IS NULL OR exers.ownerId = @ownerId)
      AND exers.exerciseld = @exerciseld
GROUP BY exers.exerciseld, exers.name, exers.description;
```

The query above is able to build an individual Exercise page when a user wants to learn more about it. We filter by the owner of the page because they may somehow request a page that they don't have access to so this would prevent them from seeing it. The reason it results in such a small query is because this is supposed to be a brand-new page with only the exercise that the user selects. The user cannot select multiple exercises at once, and this ensures that by only returning one exercise. This query meets the advanced specifications due to requiring multiple JOINS and having a GROUP BY with an aggregate for the potential of having multiple muscles tied to one exercise.



The screenshot shows a database interface with a 'Result Grid' tab. The grid has four columns: 'exerciseId', 'name', 'description', and 'muscles'. A single row is displayed with the following data: exerciseId 1, name 'Partner plank band row', description 'The partner plank band row is an abdominal exe...', and muscles '["Abdominals"]'. The interface also includes a 'Filter Rows' search bar, an 'Export' button, and a 'Wrap Cell Content' toggle.

	exerciseId	name	description	muscles
▶	1	Partner plank band row	The partner plank band row is an abdominal exe...	["Abdominals"]

Fig 16: The single row output of the exercise page query

Exercise Page Query Indexing Analysis

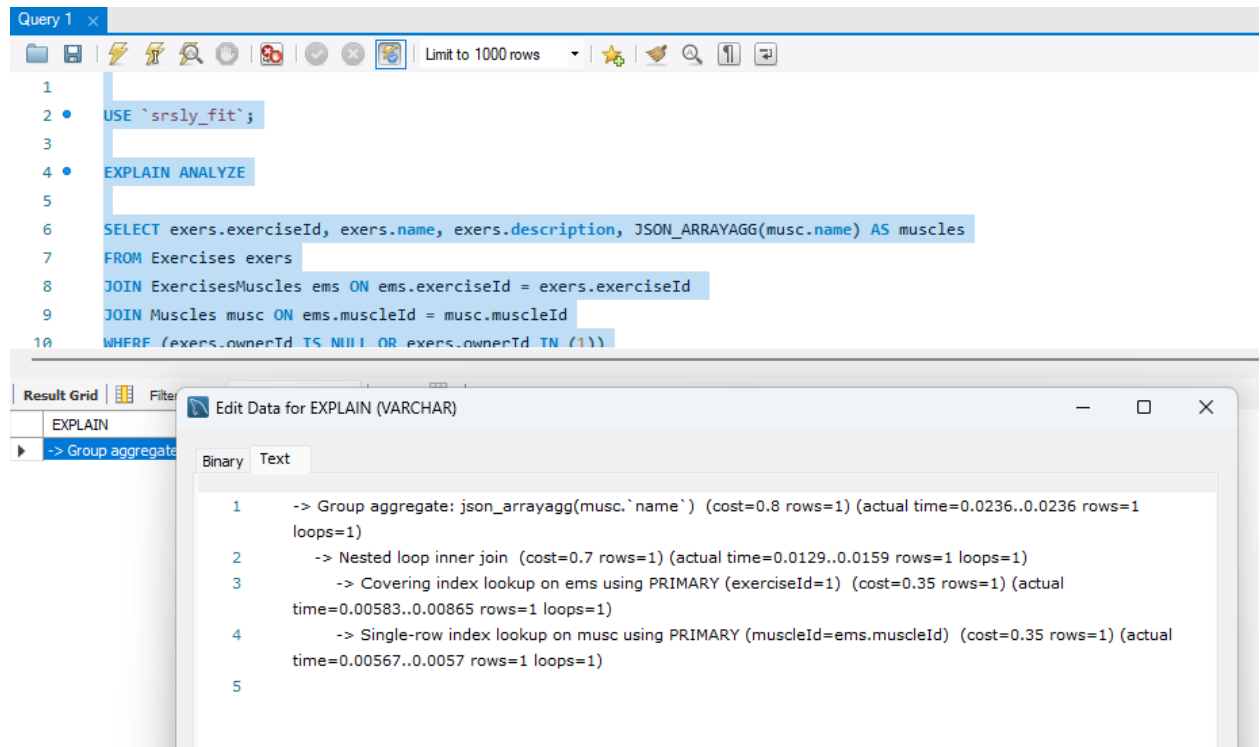


Fig 17: The EXPLAIN ANALYZE output with costs and time to complete

Indexing the above query was our next step. By avoiding primary keys (exerciseId, muscleId), only three potential attributes remained to be indexed on, with those being the exercise name, its description, and the ownerId. The default configuration (no indexes) resulted in the costs found in the table below.

Action	Cost	Rows
Group Aggregate	0.8	1
Nested loop inner join	0.77	1
Covering Index lookup on ems using PRIMARY	0.35	1
Single row index lookup on musc using PRIMARY	0.35	1

Table 4: The costs of a query run with no indexes

The first attribute we indexed was the description attribute. We used a FULL TEXT index due to the size of the description, but the results were stagnant.

Action	Cost	Rows
Group Aggregate	0.8	1
Nested loop inner join	0.77	1
Covering Index lookup on ems using PRIMARY	0.35	1
Single row index lookup on musc using PRIMARY	0.35	1

Table 5: The cost of all future indexes (as they did not decrease in cost)

The second attribute tested was the name attribute. Adding a FULL TEXT index to the name attribute did not decrease the cost. The third attribute tested was the ownerId. We used a B-Tree Index due to the fact we are parsing through simple numbers, and nothing too large. When adding this index to the previous two, we still ended in the same results. Different combinations of these also did not result in a cost decrease.

Our theory for why the cost did not decrease is because the WHERE and JOIN clauses are dependent on primary keys and unique attributes, resulting in what is already the cheapest look up time due to the uniqueness of each entry. We also think that any indexing in the GROUP BY clause would barely decrease cost as the WHERE clause has already filtered out the specific datapoints, where we are left with only one datapoint. Therefore, the GROUP BY will always be the equivalent of grouping everything by itself, making a GROUP BY check the cheapest it can be. It also is possible that a lack of data points could result in no change in the cost, and a larger data set could potentially contribute to a cost difference.

As this query is meant to only return a single result based on WHERE clause filtering, which we can confirm by the WHERE clauses uniquely identifying attributes, we can confirm that this query cannot be improved in its performance via indexing. The query is already in its most optimal state and is ready to be used.